HeatDraft ML Pipeline: Exhaustive Line-by-Line Architecture

Section 1: Initialization & Argument Parsing (Lines 1-71)

Libraries:
- Standard Python parsing, system, pathlib, and JSON libraries are loaded.
- Scientific/Data stacks: numpy, pandas, matplotlib, seaborn, rdkit (Cheminformatics).
- ML stack: Scikit-Learn (base metrics, ColumnTransformer, RidgeCV, StackingRegressor, PCA, KMeans, KFold) and Optuna for Bayesian hyperparameter tuning.
- Tree Models: XGBoost, ExtraTrees, RandomForest, and HistGradientBoosting regressors.

System Handling (Lines 30-32):
- The script detects if run on Windows and forces sys.stdout to UTF-8 encoding. This prevents execution crashes when printing greek/math symbols typical in chemical feature names (e.g. gamma).

parse_args() (Lines 35-71):
Arguments mapped:
- input_file: Positional arg for target .csv/.xlsx.
- target: Predictand column (default: Removal rate (%)).
- header: Index of spreadsheet row containing column names (default: 1).
- trials: Total optuna permutations allocated (default: 90).
- high-threshold: The target threshold defining what represents 'High Performance' (default: >90% removal).
- Feature dropping thresholds: corr-threshold (0.92), vif-threshold (10.0), nzv-threshold (0.01).
- Categorical strictness: max-cat-cardinality (60), cat-unique-ratio-threshold (0.95).
- dry-run flag: A diagnostic mode to prep data and halt before intensive ML training.

Section 2: File Detection & Loading (Lines 74-121)

auto_pick_input_file():
- Automatically crawls the directory if no file is explicitly passed. Combines globbing for .csv and .xlsx. Throws system exits if 0 or >1 candidates exist.

normalize_col_name() (Lines 95-110):
- Iterates through column names to replace erratic symbols. Chemical data frequently introduces irregular byte characters (,). Translates specific unicode keys via a dictionary map (e.g. '' -> 'gamma' equivalent, stripping strange spaces).

load_dataframe() (Lines 113-121):
- Selectively loads pandas reading routines (read_excel or read_csv) based on Path suffix. Applies normalize_col_name universally ensuring analytical formulas in downstream code won't break on dirty symbols. Fails aggressively if the target_hint does not match post-normalization.

Section 3: Cheminformatics Integration via RDKit (Lines 137-165)

add_rdkit_descriptors():
- Scans loaded columns for a case-insensitive match for the name 'SMILES'. If omitted, gracefully returns the raw DataFrame.
- If found, it creates an extensive dictionary mapped to exact RDKit descriptor functions.
- Calculates 8 physical descriptors: Exact Molecular Weight (MW), LogP (partition coefficient), Topological Polar Surface Area (TPSA), H-Bond Acceptors/Donors, Rotatable Bonds, Rings, and Fraction CSP3 (carbon sp3 ratio).
- Each SMILES string is parsed via Chem.MolFromSmiles; invalid SMILES return NaN explicitly to avoid crashing the pipeline. Automatically inserts these 8 physical descriptors as new features directly onto the DataFrame.

Section 4: Data Preprocessing & Dimensionality Reduction (Lines 168-366)

drop_near_zero_variance():
- Scales strictly numerical columns and computes column variance. Flags and drops columns where standardized variance is practically flat (< threshold).

build_correlation_clusters() & select_cluster_representative():
- Constructs a mathematical distance graph of features. Features exhibiting absolute Pearson correlation > 0.92 are pushed into a shared semantic cluster.
- Instead of randomly slicing, `select_cluster_representative` runs mutual_info_regression directly against the exact ML Target. Meaning, of the 5 highly correlated variables, it mathematically proves which 1 variable correlates most purely to 'Removal Rate' and uniquely saves that one while dropping the other 4.

drop_high_vif_features() (Lines 281-315):
- Variance Inflation Factor recursion. VIF assesses broader multicollinearity (a feature being predicted by a mixture of OTHER features, rather than just 1-to-1 correlation).
- Iteratively calculates VIF using the statsmodels formula variance_inflation_factor. Continuously drops the single worst column recursively until the entire matrix exhibits max VIF < 10.0.

full_feature_selection() (Lines 318-366):
- Orchestrator function combining Sparsity Dropping (>95% missing text), Near-Zero Variance, Correlation dropping, and VIF dropping sequentially. Compiles a detailed diagnostic JSON report tracking exactly what variables died, when, and why.

Section 5: Dynamic Sample Weighting & Target Transformations (Lines 794-822)

target_transform() (Lines 532-536):
- Removal rates operate strictly between bounds of 0% and 100%. Regressors natively do not understand bounds and will curve predictions to mathematically meaningless figures (e.g. 110%).
- Safely bounds values to a 0.0-1.0 probability, applies a clipping limit (eps=1e-4) to prevent log(0) domain errors, and computes the logit: np.log(p / (1.0 - p)).

High Zone Stratification (Line 798):

HeatDraft ML Pipeline: Exhaustive Line-by-Line Architecture

- Instead of treating all experimental rows equally, the pipeline divides data into 'Low Zone' (<90%) and 'High Zone' (>=90%).
- Dynamically derives a mathematical `weight_ratio`. If successful chemistry is rare (e.g. 5x fewer successes than failures), it inflates the multiplier scaling sample weights for the high zone dramatically. This inherently coerces the ML loss functions (MAE) to severely penalize themselves when they wrongly predict a highly-successful row.

Section 6: Optuna Hyperparameter Tuning Engine (Lines 436-529)

model_search_space() (Lines 436-489):
- Formulates massive combinatorial grids for state-of-the-art regressors.
- XGBoost: Grid searches parameters like learning_rate (log-uniformly), n_estimators [200-1200], gamma, L1 and L2 regularization (`reg_alpha`, `reg_lambda`), using the ultra-fast `hist` tree method.
- ExtraTrees & RandomForest: Grid searches n_estimators, max_depth [4-40], min_samples_split, min_samples_leaf, and max_features. Enforces criterion="absolute_error".

tune_model() (Lines 492-529):
- Employs Optuna via a dynamic maximization objective function. It spins up a KFold Generator explicitly constrained strictly to the localized scoped `X_train` data fold. This prevents data leakage heavily.
- For every single trial of hundreds of hyperparameter guesses, it transforms the subset, enforces the High-Zone penalty sample_weights, validates via Mean Absolute Error (MAE), inverses the loss, and directs Bayes Optimization towards peak convergence. It natively outputs a completely fully-fitted pipeline attached to the globally optimal configuration.

Section 7: Stacking Regressor Assembly (Lines 871-903)

- Ranks the completely tuned Base Architectures operating on the unseen 20% holdout test set using R2, RMSE, and 'High Hit Rate'.
- Compiles the absolute Top 3 performing unique models.
- Orchestrates a `StackingRegressor`. It fits all 3 top models as standard estimators, then directs their outputs directly into a unified Meta-Model (`RidgeCV`). The ridge meta-model calculates exact linear blendings bridging the slight predictive flaws of each base model, creating an ultimate overarching ensemble. This final ensemble pipeline is also trained specifically via the dynamic high-zone sample weights.

Section 8: K-Means Gated Mixture of Experts (MoE) (Lines 618-715)

A complex architecture designed to model disjoint chemical subsets.
- The data numeric structure is imputed, robustly scaled, and run through a Principal Component Analysis (PCA) to aggressively collapse sparse feature space.
- The PCA projection is fired into an unsupervised KMeans clustering algorithm (n_clusters=2), discovering distinctly partitioned physical realms of experimental data.
- The pipeline duplicates the highly-tuned generic StackingRegressor, cloning it independently onto each pure cluster.
- Inference phase (`predict()`): An incoming molecule enters the MoE. It is reduced by PCA, categorized by KMeans dynamically, and directed down an isolated execution path purely to the single 'Expert Regressor' mathematically trained

for that specific localized chemistry.

Section 9: Testing, Diagnostics, and Reporting (Lines 1041-1168)

build_low_failure_report() (Lines 571-615):
- Extracts every experiment yielding <90% performance.
- Subtracts physical feature medians (e.g. how much average pressure was applied?) spanning natively High-Performing cases vs completely Failed cases.
- Synthesizes the exact parameters responsible for inducing molecular/mechanical pipeline failure and isolates the 12 most critically divergent features.

Final Output Artifact Generation:
- Instantiates a 2x2 matplotlib GridSpec layout tracing Test-Fold predictive fidelity, scatter plotting global targets, generating residual divergence offsets, and stack bar-charting all constituent model R2 and MAE benchmarks.
- Encodes Correlation Heatmaps charting 2D matrix correlations prior to and subsequent to rigid multicollinearity enforcement.
- Dumps all metadata, best architecture JSON payloads, and image binaries natively to the `outputs/` volume via dictionary-to-JSON serialization routines.